C PROGRAMMING I

Made By : **KASHIF SAYYAD**

MODULE I — INTRODUCTION AND BASIC ELEMENTS OF C

■ An Overview of C, History of C Language, C as a Structured Language, Features of C

C is a procedural, high-level programming language which is small, fast and efficient. It was developed in **1972 at Bell Telephone Laboratories by Dennis Ritchie**, growing out of earlier languages BCPL and B, originally written to build the UNIX operating system. Later it was standardised as ANSI C (1989) and ISO C.

C as a structured language means that a C program is not written as one long jump-filled block but is divided into separate functions (modules), and each function is again divided into blocks built from sequence, selection and repetition statements only. Structured programming removes the errors of uncontrolled jumps (like a goto) and makes programs readable, testable and easier to maintain.

Features of C:

- **Simplicity, small size** — it has only about 32 keywords and is easy to learn.
- **Portability** — the same program can be run on different machines with little or no change.
- **Speed and efficiency** — compilation and execution are very fast.
- **Rich library** — a large set of library functions ranging from memory to I/Os is available.
- **Power of pointers and low-level access**, and **modularity with reusability of functions** — features that also make C a middle-level language suitable for both system and application software.

■ Basic Elements — Character Set, C Tokens, Identifiers and Keywords

Character set: every character permitted in a C program belongs to the ASCII character set — letters (A–Z, a–z), digits (0–9), special symbols and white spaces (space, tab, newline).

C Tokens are the smallest individual units of a C program. There are six types of tokens: **keywords, identifiers, constants, strings, special symbols and operators**. For example, in the statement ‘sum = a + b;’ the tokens are sum, =, a, +, b and ;.

Identifier is the name given by the programmer to a variable, array or function. Its rules are: it must begin with an alphabet or an underscore; no blank space or special symbol (except underscore) is allowed; it should not be a keyword; and it is case-sensitive, so NUMBER, number and Number are three different names.

Keywords are the words whose meanings are already fixed by the C compiler and which cannot be used as identifiers. Some of the 32 keywords are: int, char, float, double, if, else, switch, case, default, while, do, for, break, continue, goto, return, struct, union, typedef, enum, static, extern, register, auto, const, void, sizeof, long, short, signed, unsigned, volatile.

■ Variables, Constants and their Types, Operators

A **variable** is a named area of memory whose value can be changed during the execution of a program; a **constant** is a quantity whose value does not change. **Types of constants** are: **integer constant** like 10, -50 (and octal / hexadecimal forms); **floating point (real) constant** like 3.14 or 1.2e3; **character constant** like 'A' or '\n' enclosed in single quotes; and **string constant** like "HELLO" enclosed in double quotes (stored with a null character at the end).

Operators of C:

- **Arithmetic operators** — + , − , * , / and % (modulus gives the remainder).
- **Relational operators** — < , > , <= , >= , == , != which compare two values and give 0 (false) or 1 (true).
- **Logical operators** — && (AND), || (OR), ! (NOT) and are used to join relational expressions.
- **Unary operators** — unary + and −, and **increment (++) and decrement (−−)** which can be used in prefix form (change the value first) or post-fix form (use the value, then change).
- **Assignment operators** — = plus the shorthand forms += , −= , *= , /= , %= , &= , |= , ^= , <<= , >>= .
- **Conditional operator (? :)** — a ternary operator which acts as a shortened if-else, e.g. max = (a > b) ? a : b;

Precedence and associativity of operators: when an expression has several operators, **precedence** (priority) decides which operator is applied first (arithmetic before relational, relational before logical, unary strongest of all). **Associativity** decides the order when operators of the same precedence occur side by side — POOR order left to right for arithmetic, or right to left for unary and assignment.

■ Data Types, Declaration and Initialization, Type Modifiers

Data type tells the compiler what type of data a variable can hold and how much memory it needs. The basic data types are **int (integer), char (single character), float (real number of about 6 digit precision)** and **double (real number of 12 or more digit precision)**. **Declaration**: int a, b; char ch; and every variable can be given a beginning value at the time of declaration, which is called **initialisation** (int count = 0;).

Type modifiers change the size or the range of the basic types: **long** (long int, long double), **short** (short int), **signed** (both negative and positive values) and **unsigned** (only positive values, but double the maximum range). int and its modifiers combine, so short int, long int, signed int, unsigned int are all legal types of the same family.

MODULE II — C PROGRAMS AND I/O STATEMENTS, CONTROL STATEMENTS

■ Structure of C Program, Compilation and Execution

Every C program has the same general structure: off goes the documentation section, then the **link section** (include the header files with #include), the **definition section** (#define of constants and macros), the **global declaration section** (variables usable by all functions), then the **main()** function section and finally the **sub-program section** (all other user-defined functions). Execution always starts at the first statement of main().

```
#include               /* 1. Link section           */
#define PI 3.14                /* 2. Definition section      */
int counter;                   /* 3. Global declarations     */

int main(void)                 /* 4. main() function         */
{
    float radius, area;        /* 5. Local declarations      */
    printf("Enter radius : ");
    scanf("%f", &radius);
    area = PI * radius * radius;
    printf("Area = %.2f", area);
    return 0;
}
```

Compilation converts the whole source program into machine-code object file, and **linking** joins our object code with the library functions to produce the executable file, which is then loaded and **executed** by the CPU. In Turbo C, the keys are Alt+F9 (compile) and Ctrl+F9 (run); in modern systems the compiler may be gcc and the process is gcc file.c → a.out / a.exe.

■ I/O Statements, printf() / scanf(), Escape Sequence Characters

All the input and output of console works through **stdio.h**. **printf()** is a formatted output function which prints characters, numbers and the values of variables onto the screen: printf("format string", list_of_arguments);". **scanf()** is a formatted input function which reads data from the keyboard and stores it in the memory address of the given variables: scanf("%d", &age); — note the & (address-of) operator before every ordinary variable.

Format specifier	Reads / prints	Example
%d	signed decimal integer	printf("%d", n);
%f	floating point number	printf("%f", salary);
%c	single character	scanf("%c", &ch);
%s	string (till white space)	printf("%s", name);
%u / %ld / %lf	unsigned / long int / double	scanf("%ld", &big);

Escape sequence characters are the two-character sequences starting with a backslash which printf or scanf treat as single special characters: **\n** (new line), **\t** (horizontal tab), **\b** (backspace), **\r** (carriage return), **\f** (form feed), **\a** (alert bell), \\ (backslash itself), **\'** (single quote) and **\"** (double quote).

■ Library Functions — General and Maths

The **general library functions** of C header files are: from **stdio.h** — printf, scanf, getchar (reads one character), putchar (writes one character), gets (reads a whole line of text), puts (prints a string), fflush; from **conio.h** — clrscr (clears screen), getch, getche (reads a key without echo); from **stdlib.h** — exit, malloc, calloc, free,

realloc; from **string.h** — strlen, strcpy and category of the string functions; and from **ctype.h** — isalpha, isdigit, toupper, tolower.

Maths functions from **math.h** used frequently are: **sqrt(x)** (square root), **pow(x, y)** (x raised to power y), **abs(x)** (absolute value), sin(x), cos(x), tan(x), exp(x), floor(x), ceil(x) and round functions. Although declared and provided ready-made, the prototype of every library must be included through its own header file.

■ Conditional & Iterative Statements, goto

Conditional statements choose one of the available paths. **Simple if** executes a block only when the condition is true; **nested if** is an if inside another if; **if-else** chooses between two blocks; **else-if ladder** chooses one out of many conditions, checking them from the top downwards. **Switch-case statement** compares one integral expression with a list of constant **case** labels and jumps to the matching case, executes its block until break, and falls into **default** when no case matches. The **conditional expression operator (? :)** works as a single-line if-else (condition ? value_if_true : value_if_false).

Iterative (loop) statements repeat blocks of statements: **while** — first checks the condition, then executes (entry-controlled loop; may run zero times); **for loop** — initialisation, condition and increment / decrement all in one line by the syntax for (init; condition; update); **do-while** — executes the body first and then checks the condition (exit-controlled loop, executed at least once, ended by a semicolon).

break and continue: break immediately jumps out of the complete loop (or case), continue leaves only the current iteration and starts the next. The **goto statement** unconditionally jumps to a label written in the same function (goto end; ... end:;); it is allowed in C but should be used very rarely, because it destroys the structure of the program.

MODULE III — ARRAYS AND STRING FUNCTIONS

■ Arrays — Introduction, Declaration and Initialisation, Accessing Array Elements, Memory Representation

An **array** is a collection of similar data items stored under a single name in contiguous memory locations and addressed by the common name and index numbers. Its **declaration** tells the type, the name and the maximum number of elements: **int a[10];**. **Initialisation** can be done at the time of declaration (int a[5] = {10, 20, 30, 40, 50};) or by assignment later, element after element.

Accessing array elements is done with the subscript operator [ ] — the first element is a[0], the second is a[1], ... the last is a[n−1] for an array of n elements; no range checking is done by the compiler, so going beyond the array gives garbage results or crashes.

Memory representation of array: the elements are kept in consecutive bytes; **address of a[i] = base address + i × size of one element**. Because the base address is the address of a[0], the expression a[i] is calculated as *(a + i), as can also be verified from the pointer method of the next chapter.

■ One Dimensional and Multidimensional Arrays, Character Array

A **one dimensional array** is a simple sequence of elements (list of marks, list of numbers). A **two dimensional array** (matrix) is declared with two subscripts per dimension as int a[3][4] — meaning 3 rows, 4 columns — and is actually stored row-by-row (row-major) inside the memory. Three dimensional and higher arrays work in the same way; the total number of elements equals the multiplication of all the dimension sizes.

A **character array** is an array of char elements, and mostly it is used to hold a string. When a character array stores a string, the string ends with the **null character '\0'**. char name[10] = "KASHIF"; keeps six useful characters and seven bytes because the compiler adds the null automatically. Remember, an ordinary char array is not itself a string until it ends in '\0'.

■ Introduction to String and String Handling Functions

A **string** in C is an array of characters that ends in a null character '\0'. All the string functions are declared in the **string.h** header file; the commonly used ones are:

Function	Meaning	Working
strcpy(dst, src)	string copy	copies src into dst (till '\0')
strcmp(s1, s2)	string compare	0 if equal, positive / negative lexicographically otherwise
strcat(s1, s2)	concatenate	appends s2 at the end of s1
strlen(s)	string length	returns number of characters, not counting '\0'
strupr(s) / strlwr(s)	upper / lower case	converts every letter of string to capital / small
gets(s), puts(s)	input / output	reads whole line into s and prints s with newline

---